

Criterion B - Planning

Decomposing Problems

1. Database Management

- **Scheme Definition :**
 - ❖ Define the database schema for the tasks table
 - ❖ Fields : id(auto-generated), task_name, category(subject/activity), deadline, importance_level(1-5), urgency_level(1-5), time_spent(minutes)
- **CRUD Operation :**
 - ❖ Create: Add new academic or CAS tasks to the database
 - ❖ Read: Retrieve tasks to display in the main dashboard/UI, or schedule view
 - ❖ Update: Modify task details or update the “ time_spent” field
 - ❖ Delete: Remove completed or cancelled tasks from the records.
- **Sorting and Prioritization :**
 - ❖ Implement an algorithm in order to calculate a “Priority Score” based on urgency and importance.
 - ❖ Implement a sorting functionality that could sort the tasks by any field.

2. User Interface (UI)

➤ Form Design:

- **Input Fields:**

- ❖ Combo Boxes: For Category, allowing users to select from a list of their classes. For the sort button, allowing users to select between all the fields for the sorting function.
- ❖ Sliders/Spinboxes: For importance_level, and urgency_level, a slider would allow the user to select between values. For time_spent if users don't time themselves using the app, they can use a spinbox to input their time_spent.
- ❖ Date Picker: Allows users to input their deadlines.
- ❖ Text Fields: For task_name allowing users to input data manually.

- **Buttons:**

- ❖ Add: To save new entries to the database
- ❖ Edit: To modify existing entries, through input fields
- ❖ Delete: To remove entries
- ❖ Search: To search for existing entries through user input
- ❖ Statistics: To open the data visualization window
- ❖ Timer: To start the timing module, for time_spent field.
- ❖ Sort: To sort the schedule table through any field, upon clicking appears a combobox with mentioned fields.

➤ Display:

- **Schedule Table:** To view all upcoming tasks and sort them through every field.
- **Data Visualization:** To show statistics and graphical representation, using Matplotlib
- **Alert System:** Visually indicates for tasks that are "Overdue", "High Priority", or deadline is <1 day away.

3. Time tracking & Algorithmic Logic

Time Logging:

- ❖ Implement a stopwatch function using the "time" library in order to record duration spent on a task
- ❖ Automatically update the "time_spent" field in the database upon stopping, or the user starts the time module again.
- **Prioritization Engine:**
 - ❖ Create a logic function that if any alert system is active, the tasks that are alerted will appear at the top.

4. Data Visualization (Using Matplotlib)

- **Library Implementation:**
 - ❖ Utilize matplotlib.pyplot to generate static and interactive statistical charts
 - ❖ Utilize FigureCanvasTkAgg to embed dynamic charts directly into the Tkinter GUI frame, which would ensure a seamless user experience.
- **Graphing Logic:**
 - ❖ Through the concept of distribution analysis, aim to generate a pie chart using aggregated “time_spent” data grouped by “category” to visualize the total workload balance.
 - ❖ Through comparative analysis, develop a grouped bar chart in order to compare “Estimated time” versus the “Actual time spent” per category/subject, which could identify inaccuracies in the student’s organization of time.
- **Imbalance Workload Detector:**
 - ❖ Implement an algorithmic check that calculates the mean “time_spent” across all subjects
 - ❖ Trigger a visual “Imbalance Warning”, (as a 5-10s pop-up), if a single category proceeds to exceed the mean by a defined threshold,(eg., + than 40% of total weekly time)

5. Error Handling and Validation

- **Input Validation:**
 - ❖ Make sure that deadlines are not set in the past and task names are not empty
- **Error Handling:**
 - ❖ Provide user feedback on errors or issues during operations (e.g., invalid input, database errors).
 - ❖ Display pop-up for successful saves.

6. Controller Logic

- **Event Handling:**
 - ❖ Map UI actions to specific Python functions
 - ❖ Trigger prioritization systems as soon as a new task is saved/added, or an urgency level is changed in the combo box
- **Data Synchronization:**
 - ❖ Ensure “Schedule View” refreshes automatically whenever the database is updated
- **Visualization Logic**
 - ❖ Manage the communication between SQLite database and Matplotlib
 - ❖ Calculate the data subsets before passing them to the graphing function which makes sure the UI is responsive